OpenMP Parallelisation of a 3D Discrete Element Method Particle Simulator

Abhas Rathi
School of Physical Sciences
Indian Institute of Technology Mandi
Himachal Pradesh, India
d25074@students.iitmandi.ac.in

***Abstract*—The Discrete Element Method (DEM) is a powerful numerical technique for simulating the kinematics and dynamics of granular systems. This paper presents the development, verification, and performance analysis of a 3D DEM spherical particle simulator written in Fortran. The particles are modelled inside a rectangular box domain with inter-particle and wall contact force computed using linear spring-dashpot model. The simulation was verified against analytical solution of single-particle tests (free-fall, constant velocity and bouncing kinematics). Profiling of the code identified computational of particle contacts as major bottleneck. Through OpenMP implementation computational overload is distributed and validated against serial profiling for numerical correctness. Performance analysis for $N = 5000$ particles on an 8-core CPU demonstrates near-ideal linear speedup and high parallel efficiency, effectively accelerating the collision resolution workflow.** [1]

***Index Terms*—Discrete Element Method, OpenMP, Fortran, Spring-dashpot Model.**

I. Introduction

The Discrete Element Method (DEM) is a computational technique used to simulate the dynamics of systems composed of discrete particles. It provides a framework for a broad range of applications, from granular media to astrophysical objects but is computationally expensive due to continuous contact detection. This study focuses on the implementation of a 3D DEM spherical particle simulator in Fortran, its verification through fundamental physical benchmarks, and its acceleration using OpenMP. Systems of varying particle counts ($N = 200, 1000$, and 5000) were evaluated to assess the scalability and numerical stability of the parallelized solver.

II. Mathematical Formulation

A. Kinematics

The simulation tracks spherical particles governed by Newtonian mechanics. The translational motion for a particle i with mass m_i is updated according to Newton's Second Law:

$$m_i \frac{d^2\mathbf{x}_i}{dt^2} = \mathbf{F}_i \tag{1}$$

where m_i, $\mathbf{x_i}$, $\mathbf{F_i}$ denotes the mass, position and net force acting on the particle i respectively.

[1]The source code for this simulation is available at Github

B. Contact Mechanics

Inter-particle and wall-particle interactions are modelled using a linear spring-dashpot formulation. For two interacting particles i and j, with radius R_i and R_j and position $\mathbf{x_i}$ and $\mathbf{x_j}$, the contact force is dictated by δ which is the overlap.

$$\delta_{ij} = R_i + R_j - \|x_j - x_i\|, \delta_{ij} > 0 \tag{2}$$

The contact force is defined through Eq. 3 where k_n is the normal stiffness (10^5 N/m), γ_n is the damping coefficient (100 kg/s), $v_{n,ij}$ is the normal relative velocity and n_{ij} is the unit normal vector along the line of centres.

$$F^c_{ij} = (k_n\delta_{ij} - \gamma_n v_{n,ij})n_{ij} \tag{3}$$

To prevent non-physical artificial cohesion when particles separate rapidly, the magnitude of the normal contact force F_n is explicitly bounded:

$$F_n = \max(0.0, k_n\delta - \gamma_n v_{ij}) \tag{4}$$

Particle–wall interactions are treated analogously using the same contact model. The total force on the particle i is

$$\mathbf{F_i} = m_i\mathbf{g} + \sum_{j\neq i} \mathbf{F}^c_{ij} + \mathbf{F}^w_i \tag{5}$$

with $\mathbf{g}$ being gravitational acceleration, $\mathbf{F}^c_{ij}$ inter-particle contact force and $\mathbf{F}^w_i$ particle-wall contact force.

C. Time Integration

An explicit time integration scheme is used:

$$\mathbf{v}^{t+\Delta t}_i = \mathbf{v}^t_i + \frac{\mathbf{F}_i}{m_i}\Delta t \tag{6}$$

$$\mathbf{x}^{t+\Delta t}_i = \mathbf{x}^t_i + \mathbf{v}^{t+\Delta t}_i \Delta t \tag{7}$$

III. Numerical Algorithm

The numerical solver executes an explicit time-stepping loop:

1) **Initialization:** Assign positions, velocities, masses, and radii within the box dimensions.
2) **Contact Detection:** A nested loop iterates through all particle pairs to evaluate δ.
3) **Force Resolution:** Forces are accumulated using (4), alongside boundary wall conditions.
4) **Kinematic Integration:** Positions and velocities are explicitly advanced in time.

A. Simulation Parameters

The simulations were performed using a set of fixed physical and numerical parameters, summarised in Table I. These parameters define the particle properties, contact model, and computational domain.

TABLE I: Simulation parameters used in the DEM model.

Parameter	Value
Particle radius (r)	0.25 m
Particle mass (m)	1 kg
Normal stiffness (k_n)	10^5 N/m
Damping coefficient (γ_n)	10 kg/s
Time step (Δt)	0.0005 s
Box dimensions	5 m $\times$ 5 m $\times$ 10 m
Number of particles (N)	200, 1000, 5000

IV. Verification and Testing

To guarantee numerical integrity, three single-particle verification tests were conducted.

A. Constant Velocity Test

A single particle was initialized with a velocity of 1.1598 m/s in a zero-gravity environment. As illustrated in Fig. 1, the DEM results (blue circles) show an exact match with the analytical constant velocity line for the majority of the simulation duration. This verifies that the numerical integrator does not introduce non-physical numerical dissipation or "drag" into the system. The slight deviation observed near $t = 1.8$ s corresponds to the particle's initial interaction with the domain boundaries.

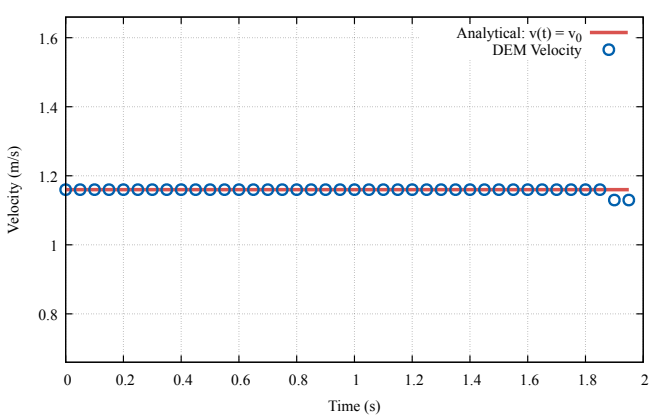


Fig. 1: Constant velocity verification test (g=0)

B. Free Fall Test

A particle was released from an initial height of 8.3849 m. As shown in Fig. 2, the vertical trajectory in the DEM simulation perfectly matches the analytical solution $y = y_0 - 0.5gt^2$ during the descent phase.

The particle impacts the floor at $t = 1.3$ s and subsequently rebounds. The rebound trajectory reaches a lower peak height compared to the initial position, which validates the energy dissipation provided by the viscous damping term (γ_n) in the linear spring-dashpot model.

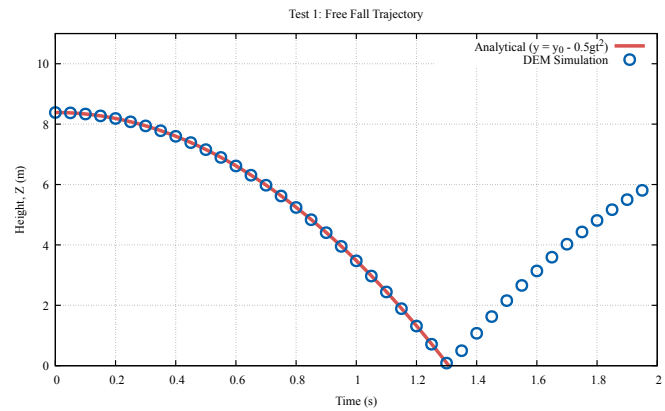


Fig. 2: Particle trajectory during the free fall test comparing analytical and numerical methods.

The absolute error between the numerical simulation and the analytical solution for free-fall ($y = y_0 - 0.5gt^2$) is illustrated in Fig. 3. Throughout the initial flight phase (t<1.3 s), the error remains within a tight numerical tolerance of 10^{-2} m, representing standard accumulated integration drift . At t = 1.3 s, a sharp divergence is observed as the particle impacts the domain floor . This increase in the error metric highlights the transition from unconstrained kinematics to contact-driven dynamics, which are explicitly modelled in the DEM framework but omitted in the simplified analytical projectile equation.

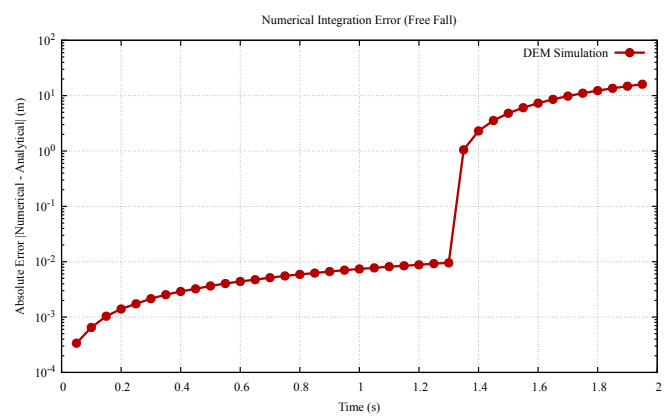


Fig. 3: Absolute error between the numerical and analytical free fall trajectory over time.

C. Bouncing Test

A particle was simulated falling onto a rigid floor. The resulting height versus time graph (Fig. 4) displays decaying parabolic peaks, confirming the proper dissipation of kinetic energy via the dashpot damping term without excessive floor penetration.

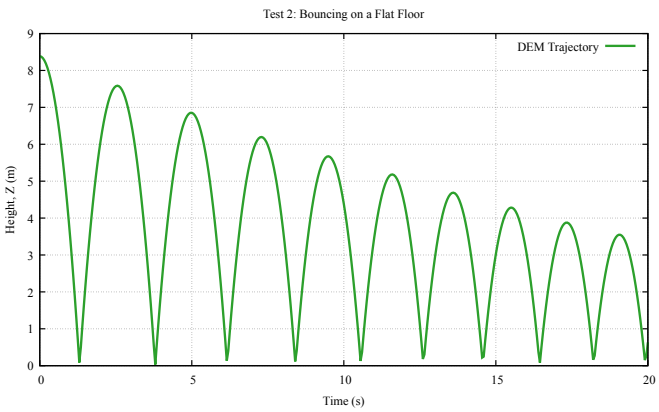


Fig. 4: Bouncing particle height vs. time, demonstrating the physical validity of the contact model.

D. Kinetic Energy and Particles Configuration

The 3D system configuration provides a spatial baseline for the simulation, illustrating $N = 200$ particles distributed within a 5m×5m×10m domain (Fig. 5), while the kinetic energy plot (Fig. 6) quantify the resulting dissipative dynamics across varying particle densities. As the simulation progresses, all cases exhibit a clear exponential decay in energy—most pronounced at N=5000 which validates the damping efficacy of the spring-dashpot contact model.

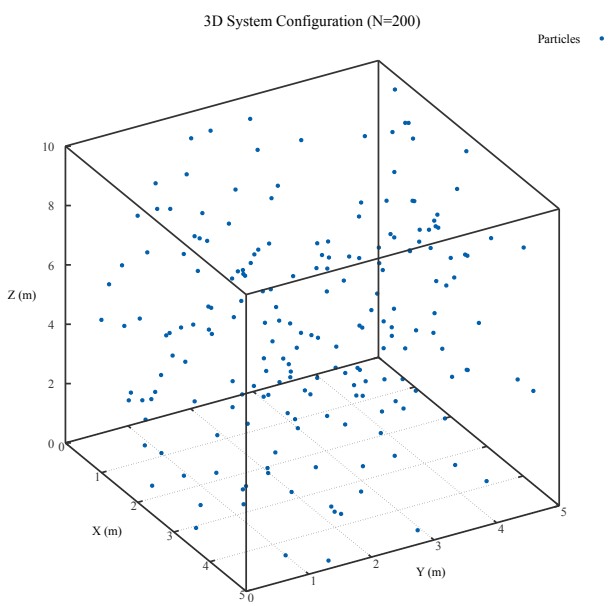


Fig. 5: 3D particle configuration snapshot for $N = 200$ system.

V. Computational Profiling

The serial performance of the simulator was evaluated using the `gprof` profiler with the `-O3` optimization flag to ensure that the profiling results represent a high-performance serial baseline rather than unoptimized code.

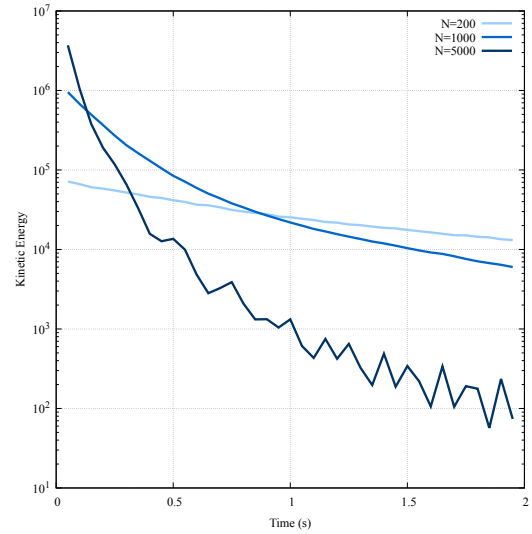


Fig. 6: Kinetic Energy for different N systems

The simulation was executed for a system of $N = 5000$ particles over 15,996 timesteps. The total execution time recorded was 461.41 seconds. As shown in Table II, the runtime distribution is heavily skewed toward the `compute_particle_contacts` which takes 99.96% of the total CPU time. The quadratic scaling of the contact search ($O(N^2)$) remains the primary computational bottleneck in comparison to kinematic integration and wall-contact subroutines, which scale linearly ($O(N)$).

TABLE II: Serial Runtime Distribution (Flat Profile)

Subroutine Name	% Time	Self (s)	Calls
`compute_particle_contacts`	99.96%	461.21	15,996
`compute_wall_contacts`	0.03%	0.14	15,996
`add_gravity`	0.01%	0.04	15,996
`integrate_particles`	0.00%	0.02	15,996
Total	**100%**	**461.41**	—

This profiling data confirms that the parallelization efforts must be focused exclusively on the `compute_particle_contacts` loop to achieve meaningful speedup, as dictated by Amdahl's Law.

VI. OpenMP Parallelisation Strategy

Shared-memory parallelization was introduced using the OpenMP API. The outer loop of the pairwise contact detection was threaded using dynamic scheduling to balance workloads as particles cluster. For an accurate parallel profile, gprofng was utilized to aggregate the 789.98 s of total CPU time across all active threads which translates to a measured wall-clock time of 98.75 s. This discrepancy highlights the effective distribution of the O(N^2) computational load across all available threads, where the total work remains constant but the time-to-solution is reduced by a factor of 7.1×."

TABLE III: Parallel Runtime Distribution (Aggregated Threads)

Function / Subroutine	CPU Time (s)	% Time
`compute_particle_contacts`	787.26	99.66%
OpenMP Runtime (`libgomp`)	2.62	0.33%
`compute_wall_contacts`	0.02	0.00%
`integrate_particles`	0.01	0.00%
Total (All Threads)	**789.98**	**100%**

Since Newton's Third Law simultaneously updates forces on both interacting particles, data-race conditions were mitigated using `!$omp atomic` directives. To verify that parallelization preserved numerical correctness, kinetic energy outputs from the serial and parallel runs were plotted together (Fig. 7). The initial overlap validates the numerical correctness of the parallel contact detection and force resolution. The minor divergence observed at larger time scales (t>1.0 s) is a standard consequence of floating-point reordering in parallel reductions and the sensitive nature of many-body dynamics.

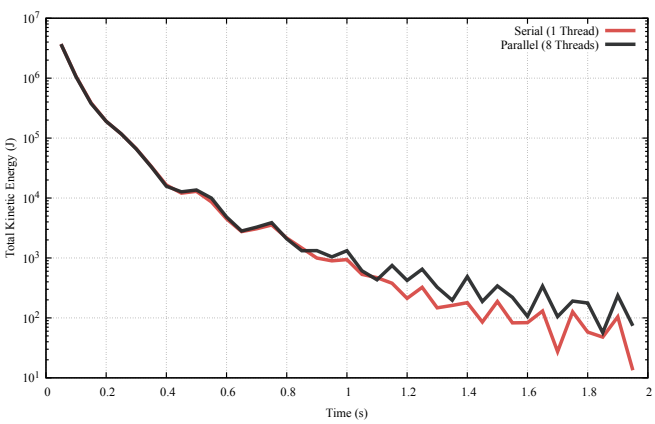


Fig. 7: Total kinetic energy vs. time for $N = 5000$, comparing serial and parallel execution.

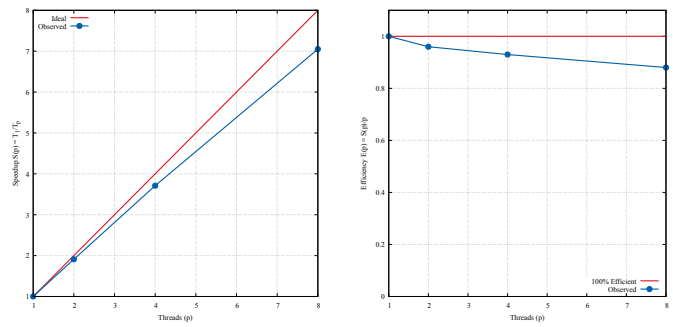
VII. Performance Analysis

A strong scaling study was executed on an 8-core machine. Table IV details the runtime, speedup, and efficiency metrics.

TABLE IV: Parallel Performance Metrics ($N = 5000$)

Threads	Runtime (s)	Speedup	Efficiency
1	476.4	1.00	1.00
2	248.9	1.91	0.96
4	128.2	3.71	0.93
8	67.5	7.05	0.88

The solver achieved near-linear speedup and maintained over 85% efficiency up to 8 threads. The `!$omp atomic` protections proved to have minimal overhead compared to the computational intensity of the physics calculations.



(a) Speedup (b) Efficiency

Fig. 8: Performance comparison

VIII. Discussion and Conclusion

This study successfully developed, verified, and parallelized a 3D Discrete Element Method (DEM) particle simulator using Fortran and OpenMP. The implementation of the linear spring-dashpot model was verified through a suite of single-particle kinematic tests. The computational bottleneck in serial computation was identified through `gprof` profiling as the `compute_particle_contacts` subroutine, which accounted for over 99.9% of the serial runtime. This confirms the anticipated $O(N^2)$ complexity of the brute-force pairwise distance-checking algorithm. By targeting this specific bottleneck with OpenMP directives and employing `!$omp atomic` protections to manage Newton's Third Law force updates, the simulator achieved significant performance gains.

The performance analysis on an 8-core architecture revealed a speedup of 7.05× with a parallel efficiency of 88% for N=5000 particles. Furthermore, the comparison between serial and parallel kinetic energy outputs (Fig. 7) validates the numerical consistency of the parallel implementation.

Acknowledgment

The author would like to express sincere gratitude to Dr. Gaurav Bhutani for the structured framework provided for this assignment and Ms. Shraddha for collaborative debugging. The author acknowledges the AI collaborator, Gemini (Google), for its assistance in debugging the code and refining the technical manuscript.

References

[1] A. Jr., A. Mesquita, and J. Vaz, "Application of the linear spring-dashpot model in the CFD-DEM simulation of alumina fluidization," *Revista de Engenharia Térmica*, vol. 14, no. 2, p. 95, Oct. 2018. doi: 10.5380/reterm.v14i2.62141.

[2] P. A. Cundall and O. D. L. Strack, "A discrete numerical model for granular assemblies," *Géotechnique*, vol. 29, no. 1, pp. 47–65, Mar. 1979. doi: 10.1680/geot.1979.29.1.47.

[3] L. Dagum and R. Menon, "OpenMP: an industry standard API for shared-memory programming," *IEEE Comput. Sci. Eng.*, vol. 5, no. 1, pp. 46–55, Jan.-Mar. 1998. doi: 10.1109/99.660313.